

America Chavez

12-8-2025

Final Project Report

The program I decided to create for my final project is a simple StoryMode game. It will run through different scenes, and in each scene checks whether a dice roll is needed. The program also has the ability for the user to choose a character, and certain characters they choose will increase their probability of surviving a dice roll later on. This program will simulate a 'roleplay'-like game, and it entertains the user with the function of choosing a character. The user can also choose different characters each time and see how that affects the gameplay. The dice roll also increases the tensions in the game, since the user has no control over whether they win or lose aside from being able to choose a certain character.

For this project, I created a few algorithms that would help the project run in its totality. They are separated into their own separate methods. Three of the algorithms that I created are: rolling the dice, printing Strings slowly, and reading information from text files.

The first algorithm was the one that rolls the dice, the code was written as the following:

```

//following method 'rolls the dice'
public int rollDice() {

    //make a new array which will hold the dice values
    int [] numbered = new int[10];

    //fill in dice values
    for (int i = 0; i < numbered.length; i++) {
        if (numbered[i] == 0) {
            numbered[i] = 1;
        }

        else {
            numbered[i] = numbered[i-1] + 1;
        }
    }

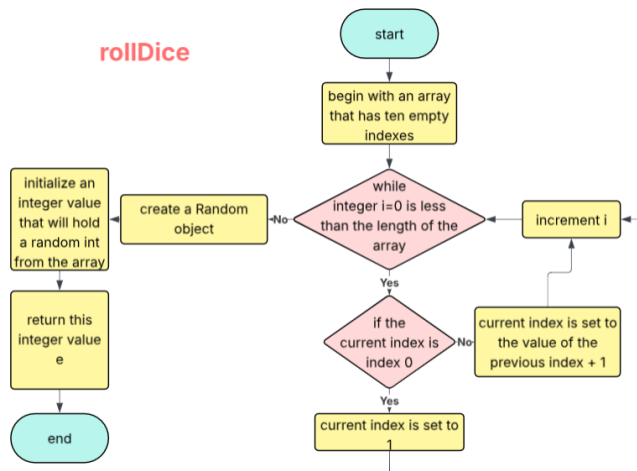
    //create an instance of 'Random'
    Random random = new Random();

    //roll the dice
    int rolled = random.nextInt(numbered.length);

    return rolled;
}
}

```

This is the flowchart that aligns with this code:



I created the dice roll as an algorithm since it would be used multiple times throughout the code, in creating an algorithm for it, I can later call on it however many times I need it and where I need it rather than to code it every time. The dice roll is used to decide whether the user ‘wins’ or ‘loses’ in certain scenes of the game.

The next algorithm is the one which reads text slowly, this is the code I partially wrote:

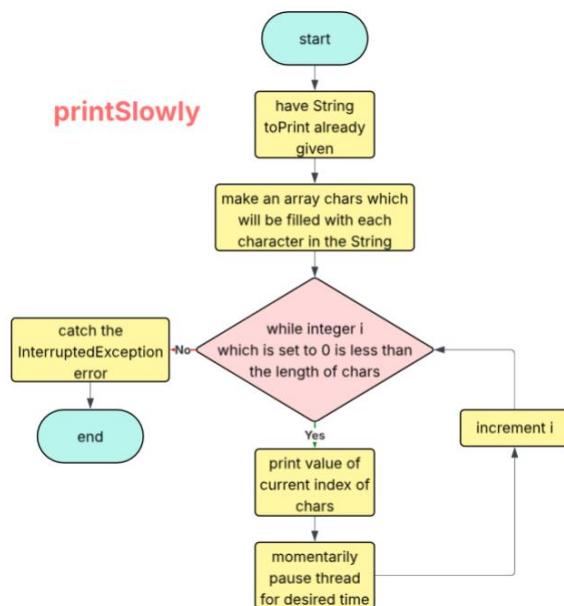
```
//following code found on StackOverflow and improved for this program
//method to print slowly for better readability
public static void printSlowly(String toPrint) {

    //separate the given string into characters and fill array
    char[] chars = toPrint.toCharArray();

    //print each character slowly
    for (int i = 0; i < chars.length; i++) {
        System.out.print(chars[i]);
        try {
            Thread.sleep(20);
        }

        //override the error that happens when thread is interrupted
        catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return;
        }
    }
}
//end of method
//end StackOverFlow code
```

The flowchart for this algorithm is the following:



I chose to use this algorithm because the program would otherwise print everything at once. This is not a problem for me since I know what each thing is doing. But for a user simply

playing the game, all of the random chunks of text would become confusing. Therefore, I felt this was needed so the user could follow the text as it was being printed.

I used the function to print slowly first when I prompted the user to choose a character, therefore they could focus on what they were being asked to do. I used it later when printing out the story. This helps the reader follow along as they ‘experience’ these scenes.

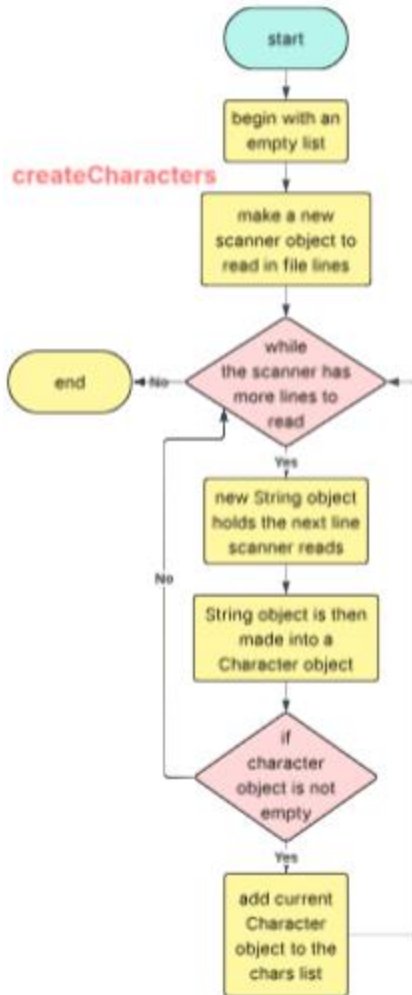
The third algorithm I created is one that reads information from text files to use as the data for characters. I wrote this code:

```
//create a list which will hold all the characters
List<Character> chars = new ArrayList<Character>();

//read in the lines
try{
    read = new Scanner(new File("characters.txt"));
    while (read.hasNextLine()) {
        String line = read.nextLine();
        //call previous method to create object
        Character character = loadCharacterFromTxt(line);
        if(character != null) {
            chars.add(character);
        }
    }
}

//throw back error if file is missing
catch (FileNotFoundException e) {
    System.out.println ("File not found: " + e.getMessage());
}
```

This is the flowchart for that algorithm:



All of these flowcharts follow the Big-O time of $O(n)$. They are all dependent on how big or small n is. For example, to fill in the dice values in the first algorithm we have to go through each one and individually fill it in. This depends on how many 'n' or indexes there are.

Moving on, I chose to use data structures throughout my program. Two of the data structures I chose were arrays and also linked lists.

I used arrays when I needed to quickly loop through sets of data using indexes, and when I was certain that the array wouldn't need to hold extreme amounts of data. For example, I used an array in one of my earlier algorithms:

```
//make a new array which will hold the dice values  
int [] numbered = new int[10];
```

I chose an array because I knew that the array for the dice would only hold ten values and because I would have to flip through each index later and fill them in with values. Afterward, I would also need to be able to flip through the array and randomly choose one of the indexes to use that value as the dice roll result.

Moving on, I used a linked list in the program whenever I wasn't sure how much data I would need to hold and needed to be able to add to it as we went. I also used them in one instance because I knew the data needed to be linked to each other in a specific order. This instance is the following:

```
//method reads in scenes from the scenes file  
public static Scene[] createScenes() {  
    Scanner read = null;  
    //create a list which will hold the scenes  
    List<Scene> scenes = new ArrayList<Scene>();
```

I used the linked list to hold the values being read into a specified order, as I needed them to stay in that order for the code to run properly. I also can't automatically assume that the files I'm reading are small, since they can be added to. So, a linked list was the option I settled with.

Following the idea of linking, I encountered the opportunity to use this concept to create the main storyline. I had struggled with the linking before this, but once I realized this I was able to link certain indexes of the story to each other and that helped my code run as I'd intended.

Likewise, I encountered many errors throughout my program. However, most of them were small errors, such as messing up indexes and such. But another instance in which I wasn't

sure what to do was when I was reading lines from the text file, and I needed to create them into objects.

I had an idea of how to solve this, such as splitting the lines into the object parts as the algorithm was reading them, but I wasn't sure how to implement it. This is when I remembered a big factor of programming is being able to talk through errors with others in order to find solutions.

Therefore, I turned to my older brother, who has been programming for years already. We talked through the problem together and settled on the fact that we could call in another method which would do the process for us of splitting it into chunks. This method would do the work for us in each line and would use an array to momentarily hold the pieces of the String before it would create a new object and use those indexes to fill in the parts of the object.

There are many things I would like to change in the next version of my game. I will settle on one for now: I want to make there be more opportunities for the user to manipulate the storyline as it occurs. For example, asking certain questions such as 'go left or right' and each choice will lead to a different outcome in the story. This would make the game more entertaining since it gives more weight to the user's impact on the storyline. At the same time, it'd make the game more entertaining to me since I'd have the creativity to write all the different storyline options, and writing is always a fun thing for me.